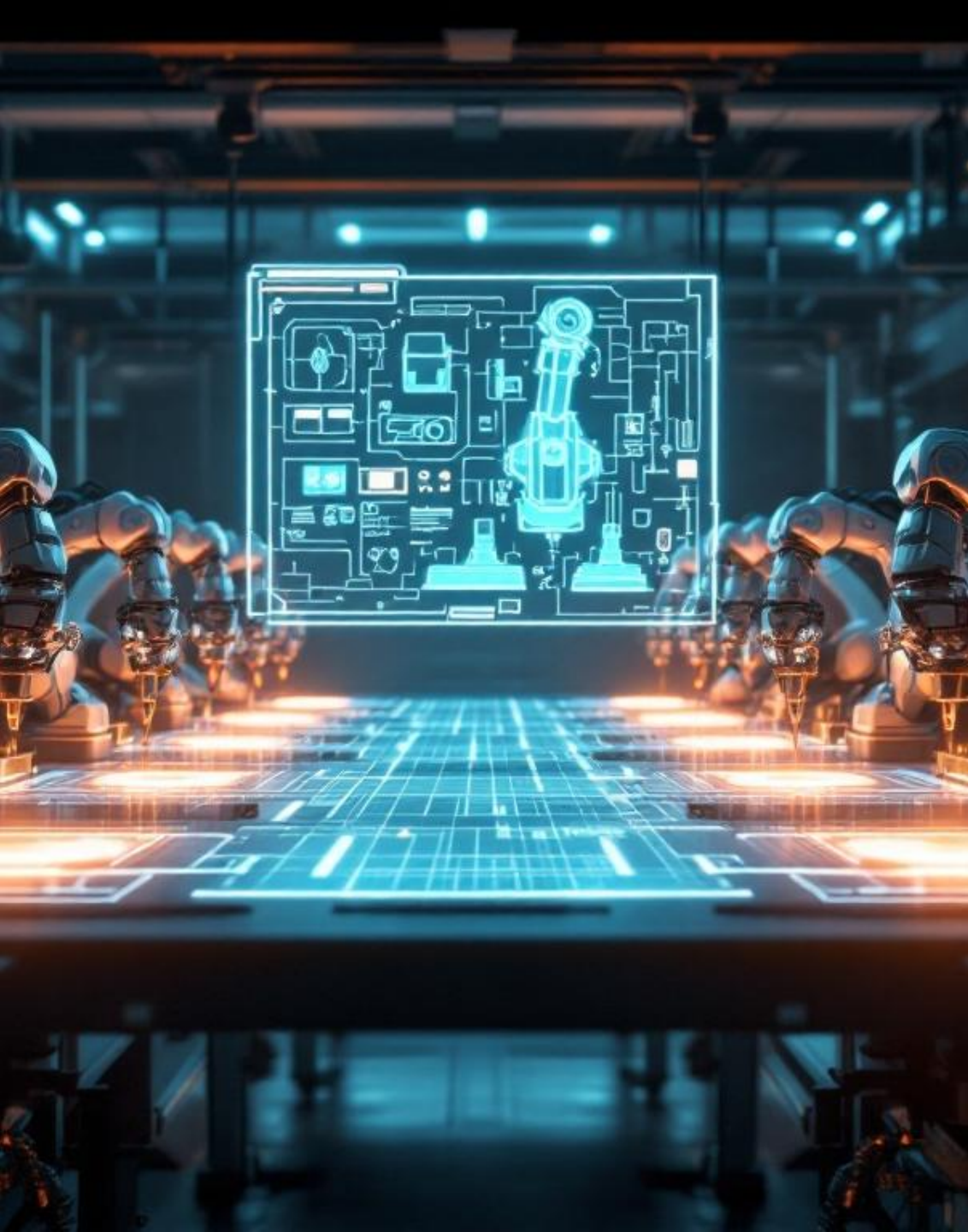




Parallel Training vs. Distributed Training

Parallelism is about how you divide the compute/parameters; distribution is about where it runs and how workers communicate and stay consistent.

- Parallelism: correctness (synchronization, gradient scaling), efficiency (load balance, communication-computation overlap).
- Distribution: network bandwidth/latency, fault tolerance, elasticity, reproducibility across nodes, cluster schedulers (Kubernetes/SLURM).
- If you scale across devices, you're doing parallel training.
- If you scale across machines (with networking concerns), you're doing distributed training. Many real systems do both.

SESSION 1: CLASSICAL DISTRIBUTED TRAINING

*In data parallelism, the **batch size** is the total number of training examples processed together before the model is updated.*

*For example, if you have a total batch size of 64 and 4 devices, each device processes a slice of 16 examples. These individual slices are often called **minibatches**.*

Distributing Training Data Across Devices

Mathematical Foundation:

- In standard SGD with batch size B , the gradient update is:

$$g = \frac{1}{|B|} \sum_{xi \in B} \nabla_{\theta} L(\theta, xi)$$

- With N devices, rank k computes:

$$g_k = \frac{1}{|B_k|} \sum_{xi \in B_k} \nabla_{\theta} L(\theta, xi)$$

- The global gradient is

$$g_{global} = \sum_{k=1}^N \frac{|B_k|}{B} g_k, \quad B = \sum_{k=1}^N |B_k|$$

- For equal local batch sizes:

$$g_{global} = \frac{1}{N} \sum_{k=1}^N g_k$$

Process Overview:

- Split the dataset into non-overlapping subsets
- Perform independent forward/backward passes
- Aggregate gradients using all-reduce
- Apply the same optimizer update on every device

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable



Reproducibility & Determinism at Scale

Seeds and sharding

- Set global and per-rank RNG seeds (Python, NumPy, framework, CUDA)
- Use a deterministic DistributedSampler so each rank gets a stable, unique shard

Deterministic ops

- Enable deterministic backends/flags
- Avoid or replace known nondeterministic kernels; document any exceptions

Floating-point non-associativity

- Different reduction orders/bucketization yield small numeric diffs
- Validate with tolerances (e.g., atol/rtol), not strict bitwise equality

Version and environment pinning

- Record container image, CUDA/cuDNN/NCCL, driver, framework versions
- Log code commit, configs/hyperparams, hardware topology and world size

Checkpoints

- Save model weights, optimizer state, RNG states (CPU/CUDA), and dataloader epoch/offset
- Include sharding metadata (DP/TP/PP groups, FSDP/ZeRO maps) for exact restarts
- Support elastic/changed world sizes with remapping info

Fault Tolerance, Checkpointing & Orchestration

Checkpointing:	- Save model, optimizer, LR scheduler, RNG states (CPU/CUDA), and sharding/group metadata - Write atomically to durable storage; verify and keep rolling N checkpoints
Failure handling:	- Synchronous jobs: relaunch and load latest checkpoint - Elastic jobs: surviving ranks continue; newcomers rendezvous, rebuild groups, and reshard from checkpoint
Launch/orchestration:	- Use SLURM/Kubernetes operators; set rank/world-size via env - Topology-aware placement (keep tightly coupled groups on same node/switch) - Add health checks, timeouts, exponential backoff, and automatic restart policies
LR & batch when elastic:	- Recompute effective global batch after world-size changes - Adjust learning rate accordingly; maintain gradient scaling and loss-scaling consistency - Log events and validate convergence after elastic resizes

Numerical Precision in AI Hardware

Trading Accuracy for Efficiency

Precision Format Evolution:

```
// Training: High precision for gradient stability
float32 weights[1024][512]; // 32-bit floating point
float32 activations[batch][1024];

// Inference: Reduced precision for efficiency
int8_t quantized_weights[1024][512]; // 8-bit integer
int8_t quantized_activations[batch][1024];

// Ultra-low power: Extreme quantization
int4_t binary_weights[1024][512]; // 4-bit or binary
```

Hardware Implications:

- **FP32**: Full accuracy, high energy cost, large storage
- **FP16/BF16**: 2x memory reduction, faster computation on modern hardware
- **INT8**: 4x memory reduction, specialized integer units, minimal accuracy loss
- **INT4/Binary**: Extreme efficiency, requires careful algorithm design

Mixed-Precision Computing:

Modern accelerators support different precisions for different operations:

- Matrix multiplication: INT8 or FP16
- Accumulation: FP32 for numerical stability
- Activation functions: INT8 for efficiency